



**UNIVERSIDAD NACIONAL AUTÓNOMA
DE MÉXICO**

FACULTAD DE INGENIERÍA

Sistemas Distribuidos



Laboratorio.

Mexi Banco con Docker Compose: lo de la clase, en tu máquina

Grupo: 02

Profesor: Oscar Manuel Ruiz Uscardo

Alumna: Segura Cedeño Luisa María

Semestre 2026-1

Fecha de Entrega: 27/09/2025

Mexi Banco con Docker Compose

Paso 1. Clonar el repo

Se realizó la descarga directa del código fuente del proyecto Mexi Banco desde GitHub a la máquina local. Utilizando el comando `git clone` con la bandera `--branch v05.1`, se especificó la versión exacta requerida, asegurando contar con la estructura base del monolito que incluye los archivos de configuración de Docker y las dependencias en Java.

Posteriormente, mediante el comando `cd mexi-banco`, se ingresó al directorio recién creado para comenzar a trabajar dentro del proyecto. Durante el proceso, Git indica un estado de *detached HEAD*, lo cual es el comportamiento normal al clonar un tag o versión específica, ya que coloca el repositorio en un punto de lectura enfocado únicamente en la versión v05.1 para su posterior despliegue con Docker Compose.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/documents/sd-2027-1 (entregas_segura_luisa)
$ git clone --branch v05.1 https://github.com/OscarRuiz21/mexi-banco.git
cd mexi-banco
Cloning into 'mexi-banco'...
remote: Enumerating objects: 123, done.
remote: Counting objects: 100% (123/123), done.
remote: Compressing objects: 100% (97/97), done.
remote: Total 123 (delta 23), reused 107 (delta 12), pack-reused 0 (from 0)
Receiving objects: 100% (123/123), 46.71 KiB | 810.00 KiB/s, done.
Resolving deltas: 100% (23/23), done.
Note: switching to '88203161aa31a371823715684a548d154e1516f8'.
```

```
You are in 'detached HEAD' state. You can look around, make experimental
changes and commit them, and you can discard any commits you make in this
state without impacting any branches by switching back to a branch.
```

```
If you want to create a new branch to retain commits you create, you may
do so (now or later) by using -c with the switch command. Example:
```

```
git switch -c <new-branch-name>
```

```
Or undo this operation with:
```

```
git switch -
```

```
Turn off this advice by setting config variable advice.detachedHead to false
```

Paso 2. Levántalo con Compose

Se ejecutó el comando `docker compose up --build -d` para construir la imagen de la aplicación y levantar la infraestructura completa definida en el archivo de composición. En la primera etapa del proceso, Docker utilizó el archivo `Dockerfile` multi-etapa para compilar el proyecto en Java con Maven. Como se observa en la terminal, la compilación finalizó de forma exitosa (BUILD SUCCESS) y empaquetó el código ejecutable en el archivo `mexi-banco-0.1.0.jar`.

Posteriormente, la segunda etapa del `Dockerfile` copió este archivo ejecutable a una imagen base ligera para generar la imagen final `mexi-banco-app:latest`. Una vez construida la imagen, Docker Compose creó automáticamente la red compartida (`mexibanco`), el volumen nombrado de datos para la base de datos (`mexi-banco_mexibanco_datos`), y desplegó en segundo plano los contenedores `mexi-banco-db-1` y `mexi-banco-app-1`. La salida confirma que la base de datos alcanzó su estado de salud óptimo (Healthy) y la aplicación inició correctamente (Started), quedando lista para recibir peticiones HTTP.

```
MINGW64/c/Users/luisa/documents/sd-2027-1/mexi-banco
#16 18.99 [INFO] Building jar: /app/target/mexi-banco-0.1.0.jar
#16 19.09 [INFO] --- spring-boot:3.3.4:repackage (repackage) @ mexi-banco ---
#16 19.83 [INFO] Replacing main artifact /app/target/mexi-banco-0.1.0.jar with repackaged archive, adding nested dependencies in BOOT-INF/.
#16 19.83 [INFO] The original artifact has been renamed to /app/target/mexi-banco-0.1.0.jar.original
#16 19.83 [INFO] BUILD SUCCESS
#16 19.83 [INFO] Total time: 16.134 s
#16 19.84 [INFO] Finished at: 2026-09-26T01:36:33Z
#16 19.84 [INFO]
#16 DONE 20.0s

#17 [stage-1 3/3] COPY --from=builder /app/target/*.jar app.jar
#17 DONE 0.1s

#18 exporting to image
#18 exporting layers
#18 exporting layers 2.0s done
#18 exporting manifest sha256:77e2fe929758ae7b55351439ef98bfe47cdcebe0fa3a12879c1ec035d7272f8d 0.0s done
#18 exporting config sha256:bef171adcf1ba7e4f7fee756c3358c520554ffca800e7e030f0c9441c5c4145 0.0s done
#18 exporting attestation manifest sha256:231f574f5b1b9482b0b8725e9eb1f7e59dc157f1f416bd0772aaadfea9ec015f 0.0s done
#18 exporting manifest list sha256:18948d6cf5ce0b553ffc60d78e7966d08e15de5c94016a2ce98a29f3d10e9ac2 0.0s done
#18 naming to docker.io/library/mexi-banco-app:latest 0.0s done
#18 unpacking to docker.io/library/mexi-banco-app:latest
#18 unpacking to docker.io/library/mexi-banco-app:latest 0.5s done
#18 DONE 2.7s

#19 resolving provenance for metadata file
#19 DONE 0.1s

Image mexi-banco-app Built
Network mexibanco Creating
Volume mexi-banco_mexibanco_datos Creating
Volume mexi-banco_mexibanco_datos Creating
Network mexibanco Creating
Volume mexi-banco_mexibanco_datos Created
Volume mexi-banco_mexibanco_datos Created
Network mexibanco Created
Network mexibanco Created
Container mexi-banco-db-1 Creating
Container mexi-banco-db-1 Created
Container mexi-banco-app-1 Creating
Container mexi-banco-app-1 Created
Container mexi-banco-db-1 Starting
Container mexi-banco-db-1 Started
Container mexi-banco-db-1 Waiting
Container mexi-banco-db-1 Healthy
Container mexi-banco-app-1 Starting
Container mexi-banco-app-1 Started

luisa@LAPTOP-S0E6SESR MINGW64 ~/documents/sd-2027-1/mexi-banco ((v05.1))
$ docker compose ps
NAME                IMAGE                COMMAND                  SERVICE    CREATED        STATUS              PORTS
mexi-banco-app-1    mexi-banco-app       "java -jar app.jar"      app        About a minute ago    Up About a minute    (healthy)    0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine   "docker-entrypoint.s..." db          About a minute ago    Up About a minute    (healthy)    5432/tcp
```

Paso 3. Prueba los endpoints

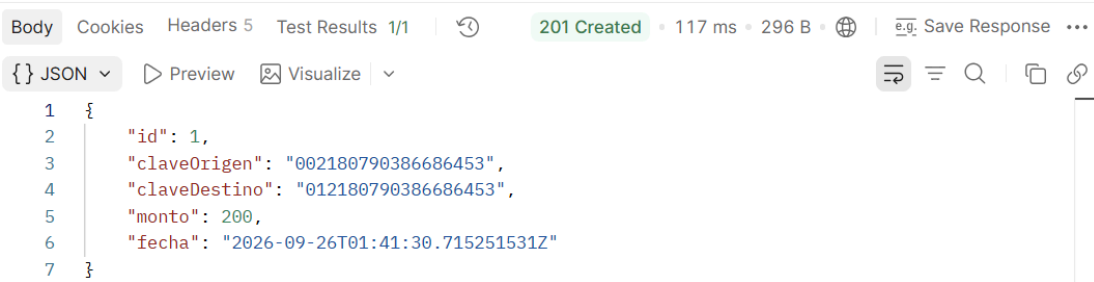
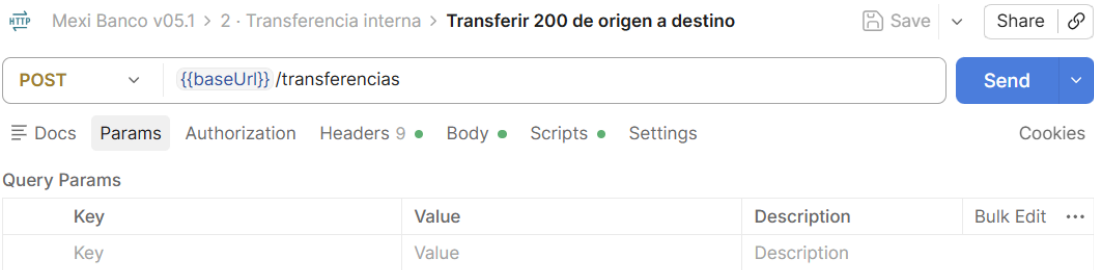
Se verificó el estado operativo de los servicios antes de comenzar con la ejecución de las pruebas HTTP en Postman. Al ejecutar el comando `docker compose ps` en la terminal, se confirmó que ambos contenedores del sistema se encuentran activos y funcionando correctamente en segundo plano.

Como se observa en la captura, tanto el contenedor de la aplicación Java (`mexi-banco-app-1`) como el de la base de datos PostgreSQL (`mexi-banco-db-1`) reportan el estado (`healthy`). Asimismo, se comprueba que el puerto 8080 del host fue mapeado con éxito hacia el contenedor de la aplicación (`0.0.0.0:8080->8080/tcp`), garantizando que la API REST del monolito está expuesta y lista para recibir las peticiones enviadas desde Postman.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/documents/sd-2027-1/mexi-banco (v05.1)
$ docker compose ps
NAME                IMAGE              COMMAND                  SERVICE   CREATED         STATUS               PORTS
mexi-banco-app-1    mexi-banco-app    "java -jar app.jar"     app       About a minute ago Up About a minute (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
mexi-banco-db-1     postgres:17-alpine "docker-entrypoint.s..." db        About a minute ago Up About a minute (healthy) 5432/tcp
```

Ejecución de: Transferir 200 de origen a destino

El servidor responde con un código de estado 201 Created, lo que confirma que la transferencia de 200 pesos se procesó e inscribió correctamente en la base de datos. En la salida en formato JSON se detallan los datos de la transacción registrada, incluyendo el ID de la operación (`id: 1`), la cuenta de origen, la cuenta de destino, el monto transferido (`monto: 200`) y la fecha exacta con la marca de tiempo.



Ejecución de: SPEI de 50 (reintento, misma clave)

En esta captura se realiza la prueba de idempotencia enviando un reintento de la petición POST al endpoint `{{baseUrl}}/spei` con la misma clave de idempotencia (`idempotencyKey`). Aunque se vuelve a enviar la solicitud, el sistema reconoce el identificador único prueba-790386686453 y responde con el mismo recurso registrado anteriormente (id: 1) con estado ENVIADO. Esto demuestra que el backend evita duplicar la transacción de 50 pesos, garantizando que una misma operación no se procese dos veces aunque la red o el cliente lo reintenten.

 Mexi Banco v05.1 > 3 · Idempotencia > **SPEI de 50 (reintento, misma clave)**

Save

Share

POST

{{baseUrl}}/spei

Send

Docs

Params

Authorization

Headers 10

Body

Scripts

Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body

Cookies

Headers 5

Test Results 2/2

201 Created

214 ms

337 B

Save Response

{ } JSON

Preview

Visualize

```
1 {
2   "id": 1,
3   "idempotencyKey": "prueba-790386686453",
4   "claveOrigen": "002180790386686453",
5   "bancoDestino": "BBVA",
6   "claveDestino": "012180000000000099",
7   "monto": 50.00,
8   "estado": "ENVIADO"
9 }
```

Ejecución de: CLABE que no existe (404)

En esta petición se evalúa la gestión de errores del sistema mediante la petición CLABE que no existe (404), Cuando algo sale mal.

Al consultar una cuenta con una CLABE inexistente (999999999999999999), la aplicación responde con el código de estado HTTP 404 Not Found. El cuerpo de la respuesta devuelve una estructura JSON estandarizada con el mensaje "No existe la CLABE 999999999999999999", confirmando que el backend controla adecuadamente las excepciones de recursos no encontrados en lugar de fallar de manera inesperada.

Mexi Banco v05.1 > 5 · Cuando algo sale mal > CLABE que no existe (404)

Save | Share

GET | {{baseUrl}}/cuentas/999999999999999999

Send

Docs | Params | Authorization | Headers 6 | Body | Scripts | Settings

Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body | Cookies | Headers 5 | Test Results 1/1

404 Not Found · 14 ms · 317 B · Save Response

Continue in Agent Mode

Diagnose | Decline

JSON | Preview | Debug with AI

```
1 {
2   "type": "about:blank",
3   "title": "Not Found",
4   "status": 404,
5   "detail": "No existe la CLABE 999999999999999999",
6   "instance": "/cuentas/999999999999999999"
7 }
```

Paso 4. Bájalos con y sin volumen

Con volumen: Al ejecutar este proceso, Docker detiene y elimina los contenedores activos (mexi-banco-app-1 y mexi-banco-db-1), pero mantiene intacto el volumen de datos (mexibanco_datos). Posteriormente, al volver a levantar el entorno, la base de datos de PostgreSQL recupera automáticamente toda la información registrada previamente en las pruebas de Postman, demostrando la persistencia exitosa del almacenamiento cuando no se destruye el volumen mapeado.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/documents/sd-2027-1/mexi-banco ((v05.1))
$ docker compose down
docker compose up -d
Container mexi-banco-app-1 Stopping
Container mexi-banco-app-1 Stopped
Container mexi-banco-app-1 Removing
Container mexi-banco-app-1 Removed
Container mexi-banco-db-1 Stopping
Container mexi-banco-db-1 Stopped
Container mexi-banco-db-1 Removing
Container mexi-banco-db-1 Removed
Network mexibanco Removing
Network mexibanco Removed
Network mexibanco Creating
Network mexibanco Creating
Network mexibanco Created
Network mexibanco Created
Container mexi-banco-db-1 Creating
Container mexi-banco-db-1 Created
Container mexi-banco-app-1 Creating
Container mexi-banco-app-1 Created
Container mexi-banco-db-1 Starting
Container mexi-banco-db-1 Started
Container mexi-banco-db-1 Waiting
Container mexi-banco-db-1 Healthy
Container mexi-banco-app-1 Starting
Container mexi-banco-app-1 Started
```

Sin volumen: A diferencia del caso anterior, el parámetro -v destruye de manera explícita el volumen nombrado de la base de datos (mexi-banco_mexibanco_datos Removing / Removed). Al volver a levantar la infraestructura, Docker genera un volumen completamente nuevo. Como resultado, la base de datos de PostgreSQL inicia desde cero sin registros previos, comprobando que la información de las cuentas y transferencias se elimina de forma permanente al remover el volumen asociado

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/documents/sd-2027-1/mexi-banco ((v05.1))
$ docker compose down -v
docker compose up -d
Container mexi-banco-app-1 Stopping
Container mexi-banco-app-1 Stopped
Container mexi-banco-app-1 Removing
Container mexi-banco-app-1 Removed
Container mexi-banco-db-1 Stopping
Container mexi-banco-db-1 Stopped
Container mexi-banco-db-1 Removing
Container mexi-banco-db-1 Removed
Network mexibanco Removing
Volume mexi-banco_mexibanco_datos Removing
Volume mexi-banco_mexibanco_datos Removed
Network mexibanco Removed
Network mexibanco Creating
Volume mexi-banco_mexibanco_datos Creating
Network mexibanco Creating
Volume mexi-banco_mexibanco_datos Creating
Volume mexi-banco_mexibanco_datos Created
Volume mexi-banco_mexibanco_datos Created
Network mexibanco Created
Network mexibanco Created
Container mexi-banco-db-1 Creating
Container mexi-banco-db-1 Created
Container mexi-banco-app-1 Creating
Container mexi-banco-app-1 Created
Container mexi-banco-db-1 Starting
Container mexi-banco-db-1 Started
Container mexi-banco-db-1 Waiting
Container mexi-banco-db-1 Healthy
Container mexi-banco-app-1 Starting
Container mexi-banco-app-1 Started
```

Paso 5. Revisa el código

En este paso abrí el proyecto en el editor de código y navegué hasta la ruta src/main/java/mx/mexibanco. Al inspeccionar los paquetes, confirmé que el sistema está diseñado como una arquitectura monolítica dividida en 5 módulos principales de negocio: cuenta, movimiento, transferencia, notificacion y spei. Cada uno de estos módulos maneja su propia responsabilidad dentro del mismo sistema:

- **Capa Controller (Controladores REST):** Define los endpoints expuestos que reciben los datos desde Postman. Por ejemplo, en el módulo cuenta gestiona /cuentas (para crear y consultar saldo con códigos 201 o 200), en transferencia expone /transferencias, y en spei expone /spei.
- **Capa Service (Lógica de Negocio):** Procesa las reglas financieras. Aquí se valida que una cuenta no se duplique (409 Conflict), que los campos requeridos existan (400 Bad Request), que no se transfiera a la misma cuenta o con saldo insuficiente (422 Unprocessable Entity), y se implementa la lógica de idempotencia registrando la cabecera Idempotency-Key para evitar cargos dobles.
- **Capa Repository:** Encargada de realizar las consultas e inserciones SQL en la base de datos PostgreSQL a través de JPA/Hibernate.
- **Capa Entity (Entidades):** Representa las tablas en la base de datos. En esta capa residen los modelos para persistir la información de las cuentas, las transferencias ejecutadas, el historial de movimientos y los registros de notificaciones o avisos.

Además de la lógica principal, analicé cómo interactúan estos módulos internamente. Por ejemplo, cuando se realiza una transferencia o un SPEI exitoso, los módulos de movimiento y notificación registran de forma automática el estado de cuenta y generan los avisos correspondientes para la CLABE de origen. Toda esta estructura de capas nos permite entender el monolito actual que descomponemos en microservicios independientes.

Paso 6. Estudia el compose y el Dockerfile

En este paso me dediqué a revisar a fondo los dos archivos principales de la infraestructura del proyecto: el Dockerfile y el docker-compose.yml, los cuales sirven para empaquetar la aplicación Spring Boot y orquestarla junto con su base de datos PostgreSQL.

Dockerfile: Muestra el contenido del Dockerfile desplegado en la terminal con el comando cat Dockerfile. En él se observa el proceso de construcción en dos etapas (*multi-stage build*) para generar la imagen de la aplicación.

- **Etapas 1 (Construcción):** Se utiliza la imagen eclipse-temurin:21-jdk-alpine que contiene el Kit de Desarrollo de Java (JDK 21). Se establece /app como directorio de trabajo, se copian los archivos del proyecto (pom.xml, el wrapper de Maven ./mvnw y .mvn) y se descargan las dependencias en

caché mediante `./mvnw dependency:go-offline -B`. Posteriormente, se copia el código fuente (src) y se compila el ejecutable con `./mvnw clean package -DskipTests`.

- Etapla 2 (Ejecución): Para no arrastrar herramientas innecesarias de compilación, se pasa a una imagen mucho más ligera: `eclipse-temurin:21-jre-alpine` (solo el entorno de ejecución JRE). Se copia únicamente el archivo `.jar` generado en la primera etapa (`--from=builder`), se expone el puerto 8080 y se establece el punto de entrada con `ENTRYPOINT ["java", "-jar", "app.jar"]` para iniciar la aplicación.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/documents/sd-2027-1/mexi-banco ((v05.1))
$ cat Dockerfile
# ---- ETAPA 1: CONSTRUCCION ----
FROM eclipse-temurin:21-jdk-alpine AS builder
WORKDIR /app
COPY pom.xml .
COPY mvnw .
COPY .mvn .mvn
RUN ./mvnw dependency:go-offline -B
COPY src ./src
RUN ./mvnw clean package -DskipTests

# ---- ETAPA 2: EJECUCION ----
FROM eclipse-temurin:21-jre-alpine
WORKDIR /app
COPY --from=builder /app/target/*.jar app.jar
EXPOSE 8080

ENTRYPOINT ["java", "-jar", "app.jar"]
```

docker-compose.yml: Muestra el contenido del archivo `docker-compose.yml` desplegado mediante el comando `cat docker-compose.yml`. Incluye comentarios explicativos sobre la resolución de nombres vía DNS embebido, la gestión de dependencias entre servicios y la persistencia de datos mediante volúmenes.

- Servicio db: Utiliza la imagen `postgres:17-alpine`. Configura el nombre de la base de datos, usuario y contraseña como `mexibanco` mediante variables de entorno. Para evitar pérdida de información, vincula la base de datos al volumen nombrado `mexibanco_datos`. Además, incluye un *healthcheck* que ejecuta `pg_isready` cada 5 segundos para comprobar que PostgreSQL esté verdaderamente listo para recibir conexiones.
- Servicio app: Se construye a partir del Dockerfile del directorio actual (`build: .`) y mapea el puerto 8080 local con el del contenedor. Se conecta a la base de datos haciendo referencia al host

db (aprovechando el DNS interno de Docker). La clave de este servicio es la instrucción `depends_on` con la condición `service_healthy`, la cual le exige a Spring Boot esperar hasta que el *healthcheck* de la base de datos dé respuesta positiva. También incluye su propio *healthcheck* consultando el endpoint `/actuator/health`.

- Redes y Volúmenes: Se define una red por defecto llamada `mexibanco` para la comunicación aislada entre los contenedores y el volumen `mexibanco_datos` para la persistencia.

```
luisa@LAPTOP-S0E65ESR MINGW64 ~/documents/sd-2027-1/mexi-banco ((v05.1))
$ cat docker-compose.yml
# Con un solo comando: docker compose up --build
# Lo que van a ver correr (S05, "de compose a orquestacion"):
# - `app` resuelve `db` por nombre (DNS embebido de Compose), sin ninguna IP en ningun lado.
# - `app` espera a que `db` este listo de verdad (depends_on + healthcheck), no solo que exista.
# - Las variables de entorno son la configuracion (factor III, ya visto en S04).
# - `db` persiste en un volumen nombrado: si tiran el contenedor y lo vuelven a subir, los
#   datos siguen ahi; si tiran el volumen, no.
services:
  db:
    image: postgres:17-alpine
    environment:
      POSTGRES_DB: mexibanco
      POSTGRES_USER: mexibanco
      POSTGRES_PASSWORD: mexibanco
    volumes:
      - mexibanco_datos:/var/lib/postgresql/data
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U mexibanco -d mexibanco"]
      interval: 5s
      timeout: 3s
      retries: 5

  app:
    build: .
    ports:
      - "8080:8080"
    environment:
      DB_HOST: db
      DB_PORT: "5432"
      DB_NAME: mexibanco
      DB_USER: mexibanco
      DB_PASSWORD: mexibanco
    depends_on:
      db:
        condition: service_healthy
    healthcheck:
      test: ["CMD-SHELL", "wget -qO- http://localhost:8080/actuator/health | grep -q UP || exit 1"]
      interval: 10s
      timeout: 3s
      retries: 5

networks:
  default:
    name: mexibanco

volumes:
  mexibanco_datos:
```

Conclusiones

A lo largo de esta práctica logré entender y poner en marcha la arquitectura completa del monolito de MexiBanco. Al iniciar configurando la base de datos de manera local y ejecutar las migraciones, me di cuenta de cómo se estructuran las tablas desde cero y lo importante que es tener un control de versiones sobre la base de datos para garantizar que el esquema se cree de forma ordenada y sin errores.

Posteriormente, al empaquetar y levantar la aplicación para probar los endpoints en Postman, pude comprobar el comportamiento real del sistema. Fue muy interesante observar cómo interactúan las distintas capas del código (controllers, services, repositories y entidades) para aplicar las reglas de negocio, validar datos, manejar errores HTTP específicos como saldos insuficientes o duplicados, y gestionar la idempotencia en las transacciones financieras.

Finalmente, la parte de containerización con Docker y Docker Compose me permitió ver el valor de aislar la aplicación y la base de datos en entornos independientes pero interconectados. Entender cómo funciona la construcción en dos etapas (*multi-stage build*) del Dockerfile para generar imágenes ligeras, sumado a la orquestación del docker-compose.yml utilizando el DNS interno y las revisiones de salud (*healthchecks*), me dejó muy claro cómo se preparan y despliegan este tipo de sistemas monolíticos en un entorno real antes de dar el paso hacia una arquitectura de microservicios.